

Terraform + Ansible + Packer Lab Guide

The Essential 20% for 80% Results

Target Audience: Experienced Linux/KVM administrators building cybersecurity labs

Platform: Ubuntu Desktop with KVM/QEMU

Focus: Fast, repeatable lab deployments with infrastructure-as-code

Table of Contents

1. [Quick Start: 15-Minute Setup](#)
 2. [Installation & Configuration](#)
 3. [Core Concepts](#)
 4. [Terraform Essentials](#)
 5. [Packer Essentials](#)
 6. [Ansible Integration](#)
 7. [Lab Management Workflows](#)
 8. [Real-World Lab Examples](#)
 9. [Troubleshooting](#)
 10. [Cheat Sheet](#)
-

Quick Start: 15-Minute Setup

Prerequisites Check

```
# Verify KVM is working
virsh list --all
sudo systemctl status libvirtd

# Check your resources
```

```
lscpu | grep -E '^CPU\(s\)|^Model name'
free -h
```

Install Everything

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install dependencies
sudo apt install -y \
    qemu-kvm libvirt-daemon-system libvirt-clients bridge-utils \
    mkisofs cloud-image-utils wget unzip git

# Install Terraform
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com/ $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install terraform

# Install Packer
sudo apt install packer

# Install Ansible
sudo apt install -y software-properties-common
sudo add-apt-repository --yes --update ppa:ansible/ansible
sudo apt install -y ansible

# Install Terraform libvirt provider dependencies
sudo apt install -y libvirt-dev

# Verify installations
terraform version
packer version
ansible --version
```

Configure Libvirt for Your User

```
# Add yourself to libvirt groups
sudo usermod -aG libvirt $USER
sudo usermod -aG kvm $USER

# Apply group changes (or logout/login)
newgrp libvirt
```

```
# Test access
virsh list --all
```

Create Workspace

```
mkdir -p ~/labs/{terraform,ansible,packer,images}
cd ~/labs
```

Installation & Configuration

Terraform Libvirt Provider Setup

Install the provider:

```
# Create plugins directory
mkdir -p ~/.terraform.d/plugins

# The provider will auto-install on first terraform init
# But you can verify it's available:
terraform version # Shows provider plugins after init
```

Configure Terraform for libvirt:

Create `~/.terraformrc`:

```
# Optional: Configure plugin cache to save bandwidth
plugin_cache_dir = "$HOME/.terraform.d/plugin-cache"
```

Create plugin cache directory:

```
mkdir -p ~/.terraform.d/plugin-cache
```

Packer Configuration

Create Packer variables file:

~/labs/packer/variables.pkr.hcl :

```
variable "vm_name" {
  type    = string
  default = "ubuntu-base"
}

variable "cpus" {
  type    = number
  default = 2
}

variable "memory" {
  type    = number
  default = 2048
}

variable "disk_size" {
  type    = string
  default = "20G"
}
```

Ansible Configuration

Create ansible.cfg:

~/labs/ansible/ansible.cfg :

```
[defaults]
inventory = ./inventory
host_key_checking = False
retry_files_enabled = False
roles_path = ./roles
collections_paths = ./collections
interpreter_python = auto_silent

[privilege_escalation]
become = True
become_method = sudo
become_user = root
become_ask_pass = False

[ssh_connection]
```

```
pipelining = True
ssh_args = -o ControlMaster=auto -o ControlPersist=60s -o StrictHostKeyChecking=no
```

Core Concepts

The Workflow Pipeline

LAB BUILD WORKFLOW

1. PACKER: Build base images (once or occasionally)
↳ Creates: ubuntu-base.qcow2, kerberos-base.qcow2, etc.
2. TERRAFORM: Provision infrastructure (VMs, networks, storage)
↳ Creates: VMs, networks, volumes using base images
3. ANSIBLE: Configure systems (software, services, settings)
↳ Configures: Kerberos, security tools, applications
4. TEST: Validate lab functionality
5. DESTROY & REBUILD: terraform destroy && terraform apply
↳ Back to working state in minutes

Key Relationships

Packer → Terraform:

- Packer creates `.qcow2` images stored in `/var/lib/libvirt/images/` or custom path
- Terraform references these images as base for VMs
- Terraform clones the base image for each VM (copy-on-write)

Terraform → Ansible:

- Terraform provisions VMs and outputs IP addresses
- Ansible uses these IPs to configure the VMs
- Can integrate: Terraform calls Ansible automatically after provisioning

Infrastructure State:

- Terraform tracks infrastructure state in `terraform.tfstate`

- This file is CRITICAL - it maps your code to actual resources
 - Never edit manually, never delete (or you lose control)
-

Terraform Essentials

Project Structure (Best Practice)

```
~/labs/terraform/  
├── modules/                                # Reusable components  
│   ├── base-vm/                          # Standard VM module  
│   │   ├── main.tf  
│   │   ├── variables.tf  
│   │   └── outputs.tf  
│   └── isolated-network/                # Network module  
│       ├── main.tf  
│       ├── variables.tf  
│       └── outputs.tf  
├── kerberos-lab/                        # Lab-specific configs  
│   ├── main.tf                          # Main configuration  
│   ├── variables.tf                     # Input variables  
│   ├── outputs.tf                       # Output values  
│   ├── terraform.tfvars                 # Variable values (gitignored)  
│   └── README.md  
├── ai-lab/  
├── cybersecurity-lab/  
└── secdevops-lab/
```

Basic Terraform Commands

```
# Initialize project (run once per project directory)  
terraform init  
  
# Validate syntax  
terraform validate  
  
# Show execution plan (what will change)  
terraform plan  
  
# Apply changes (create/modify/delete resources)
```

```
terraform apply

# Apply without interactive approval
terraform apply -auto-approve

# Show current state
terraform show

# List all resources in state
terraform state list

# Get specific resource details
terraform state show libvirt_domain.vm_name

# Destroy specific resource
terraform destroy -target=libvirt_domain.vm_name

# Destroy everything
terraform destroy

# Format code (style standardization)
terraform fmt -recursive

# Update state with real infrastructure (reconcile drift)
terraform refresh

# Output variables
terraform output
terraform output -json
```

Provider Configuration

Required in every project's `main.tf` :

```
terraform {
  required_version = ">= 1.0"
  required_providers {
    libvirt = {
      source  = "dmacvicar/libvirt"
      version = "~> 0.7"
    }
  }
}
```

```
provider "libvirt" {  
  uri = "qemu:///system" # System-level libvirt  
  # uri = "qemu:///session" # User-level (less common)  
}
```

How Terraform Consolidates VMs in a Lab

Method 1: Multiple Resources in One File

```
# kerberos-lab/main.tf  
  
# All VMs in same file, same network  
resource "libvirt_domain" "kdc" {  
  name     = "kerberos-kdc"  
  memory   = "2048"  
  vcpu     = 2  
  
  network_interface {  
    network_id = libvirt_network.kerberos_net.id  
  }  
  
  disk {  
    volume_id = libvirt_volume.kdc_disk.id  
  }  
}  
  
resource "libvirt_domain" "client1" {  
  name     = "kerberos-client1"  
  memory   = "2048"  
  vcpu     = 2  
  
  network_interface {  
    network_id = libvirt_network.kerberos_net.id # SAME NETWORK  
  }  
  
  disk {  
    volume_id = libvirt_volume.client1_disk.id  
  }  
}  
  
resource "libvirt_domain" "client2" {  
  name     = "kerberos-client2"  
  memory   = "2048"  
  vcpu     = 2
```

```

network_interface {
    network_id = libvirt_network.kerberos_net.id # SAME NETWORK
}

disk {
    volume_id = libvirt_volume.client2_disk.id
}
}

# Shared network definition
resource "libvirt_network" "kerberos_net" {
    name      = "kerberos_network"
    mode      = "nat"
    addresses = ["192.168.100.0/24"]
    autostart = true
}

```

Method 2: Using count for Similar VMs

```

# Create 3 identical client VMs
resource "libvirt_domain" "clients" {
    count = 3
    name  = "kerberos-client${count.index + 1}"
    memory = "2048"
    vcpu   = 2

    network_interface {
        network_id = libvirt_network.kerberos_net.id
    }

    disk {
        volume_id = libvirt_volume.client_disks[count.index].id
    }
}

resource "libvirt_volume" "client_disks" {
    count = 3
    name  = "kerberos-client${count.index + 1}.qcow2"
    pool  = "default"
    source = "/var/lib/libvirt/images/ubuntu-base.qcow2"
    format = "qcow2"
}

```

Method 3: Using for_each for Different VM Types

```
locals {
  vms = {
    kdc = {
      memory = 4096
      vcpu    = 2
      ip      = "192.168.100.10"
    }
    app_server = {
      memory = 2048
      vcpu    = 2
      ip      = "192.168.100.20"
    }
    db_server = {
      memory = 4096
      vcpu    = 4
      ip      = "192.168.100.30"
    }
    client = {
      memory = 2048
      vcpu    = 2
      ip      = "192.168.100.40"
    }
  }
}

resource "libvirt_domain" "lab_vms" {
  for_each = local.vms

  name      = "kerberos-${each.key}"
  memory    = each.value.memory
  vcpu      = each.value.vcpu

  network_interface {
    network_id      = libvirt_network.kerberos_net.id
    addresses       = [each.value.ip]
    wait_for_lease = true
  }

  disk {
    volume_id = libvirt_volume.lab_disks[each.key].id
  }
}
```

Creating Isolated Labs

Complete Isolated Network Example:

```
# Each lab gets its own network with unique subnet

# Kerberos Lab Network
resource "libvirt_network" "kerberos_net" {
  name      = "kerberos_network"
  mode      = "nat"                # Isolated, with NAT for internet
  domain    = "kerberos.lab"
  addresses = ["192.168.100.0/24"] # Unique subnet

  autostart = true

  dns {
    enabled = true
  }

  # Optional: DHCP reservations
  dhcp {
    enabled = true
  }
}

# Cybersecurity Lab Network
resource "libvirt_network" "cybersec_net" {
  name      = "cybersec_network"
  mode      = "nat"
  domain    = "cybersec.lab"
  addresses = ["192.168.101.0/24"] # Different subnet

  autostart = true
}

# AI Lab Network
resource "libvirt_network" "ai_net" {
  name      = "ai_network"
  mode      = "nat"
  domain    = "ai.lab"
  addresses = ["192.168.102.0/24"] # Different subnet

  autostart = true
}
```

```
# Completely isolated network (no internet)
resource "libvirt_network" "isolated_net" {
  name      = "isolated_network"
  mode      = "none"                # No external connectivity
  domain    = "isolated.lab"
  addresses = ["10.0.100.0/24"]

  autostart = true
}
```

Network Mode Options:

- `mode = "nat"` - Isolated network with internet via NAT
- `mode = "route"` - Routed network (requires host routing setup)
- `mode = "bridge"` - Bridged to physical network
- `mode = "none"` - Completely isolated, no external access

Assigning/Removing VMs to/from Labs

Add a VM to Existing Lab:

1. Edit your lab's `main.tf` :

```
# Add new VM resource
resource "libvirt_domain" "new_vm" {
  name      = "kerberos-new-server"
  memory    = "2048"
  vcpu      = 2

  network_interface {
    network_id = libvirt_network.kerberos_net.id # Uses existing network
  }

  disk {
    volume_id = libvirt_volume.new_vm_disk.id
  }
}

# Add corresponding volume
resource "libvirt_volume" "new_vm_disk" {
  name      = "kerberos-new-server.qcow2"
  pool      = "default"
  source    = "/var/lib/libvirt/images/ubuntu-base.qcow2"
}
```

```
format = "qcow2"
}
```

1. Apply changes:

```
terraform apply
# Only the new VM will be created, existing ones untouched
```

Remove a VM from Lab:

1. Delete the resource block from `main.tf`
2. Apply changes:

```
terraform apply
# Terraform will detect the missing resource and remove the VM
```

OR use targeted destroy:

```
# Remove specific VM without touching rest of lab
terraform destroy -target=libvirt_domain.new_vm -target=libvirt_volume.new_vm_disk
```

Move VM to Different Lab:

1. Remove from old lab config
2. Add to new lab config (change network_id to new network)
3. This will destroy in old location and recreate in new
4. Data will be lost unless you backup the disk first

Backup disk before move:

```
virsh shutdown vm_name
sudo cp /var/lib/libvirt/images/vm_disk.qcow2 /backup/vm_disk.qcow2
# Now proceed with terraform changes
```

Lab Power Management

Shut Down Lab (Without Destroying):

Terraform doesn't natively support "stop" state. You have two options:

Option 1: Use virsh directly

```
# Shut down all VMs in a lab
for vm in $(virsh list --name | grep kerberos); do
    virsh shutdown $vm
done

# Or force power off
for vm in $(virsh list --name | grep kerberos); do
    virsh destroy $vm # Note: "destroy" means power off, not delete
done

# Verify
virsh list --all | grep kerberos
```

Option 2: Use Terraform with running flag (Advanced)

Add to your VM configuration:

```
resource "libvirt_domain" "vm" {
  name      = "my-vm"
  memory    = "2048"
  vcpu      = 2

  running = var.vm_running # Control running state

  # ... rest of config
}
```

In `variables.tf` :

```
variable "vm_running" {
  type      = bool
  default   = true
}
```

Shut down:

```
terraform apply -var="vm_running=false"
```

Start Lab That Was Shut Down:

Using `virsh`:

```
# Start all VMs in a lab
for vm in $(virsh list --all --name | grep kerberos); do
    virsh start $vm
done

# Start specific VM
virsh start kerberos-kdc
```

Using Terraform (if you used running flag):

```
terraform apply -var="vm_running=true"
```

Auto-start Configuration:

```
resource "libvirt_domain" "vm" {
  name      = "my-vm"
  memory    = "2048"
  vcpu      = 2
  autostart = true # Start automatically when libvirtd starts

  # ... rest of config
}
```

How Terraform Sees Packer Images

The Complete Picture:

1. Packer builds image:

```
cd ~/labs/packer
packer build ubuntu-base.pkr.hcl
# Outputs: /var/lib/libvirt/images/ubuntu-base.qcow2
```

1. Terraform references the image:

```
resource "libvirt_volume" "vm_disk" {
  name      = "my-vm-disk.qcow2"
  pool      = "default" # Usually /var/lib/libvirt/images/
  source    = "/var/lib/libvirt/images/ubuntu-base.qcow2" # Packer's output
}
```

```
format = "qcow2"
}
```

What happens:

- Terraform creates a NEW volume by CLONING the Packer image
- Each VM gets its own copy (copy-on-write for efficiency)
- Original Packer image remains untouched
- You can create 10 VMs from 1 Packer image

Custom Image Location:

```
# If you store Packer images elsewhere
resource "libvirt_volume" "vm_disk" {
  name     = "my-vm-disk.qcow2"
  pool     = "default"
  source   = "/home/emerson/labs/images/ubuntu-base.qcow2"
  format   = "qcow2"
}
```

Using Packer with Different Pools:

```
# Create custom storage pool
resource "libvirt_pool" "lab_images" {
  name = "lab_images"
  type = "dir"
  path = "/home/emerson/labs/images"
}

# Reference image from custom pool
resource "libvirt_volume" "vm_disk" {
  name     = "my-vm-disk.qcow2"
  pool     = libvirt_pool.lab_images.name
  source   = "/home/emerson/labs/images/ubuntu-base.qcow2"
  format   = "qcow2"
}
```

Check Available Images:

```
# List all volumes
virsh vol-list default
```

```
# Get image details
qemu-img info /var/lib/libvirt/images/ubuntu-base.qcow2
```

Packer Essentials

Why Use Packer?

Without Packer:

- Terraform creates VM from Ubuntu Cloud Image (~5 min)
- Ansible installs packages, configures system (~10 min)
- Total: 15 minutes per VM

With Packer:

- Packer creates pre-configured base image once (~20 min)
- Terraform creates VM from Packer image (~2 min)
- Ansible applies minimal config (~2 min)
- Total: 4 minutes per VM

Multiply this by 4 labs × multiple rebuilds = massive time savings

Basic Packer Template

Simple Ubuntu Base Image:

`~/labs/packer/ubuntu-base.pkr.hcl` :

```
packer {
  required_plugins {
    qemu = {
      version = ">= 1.0.0"
      source  = "github.com/hashicorp/qemu"
    }
  }
}

variable "vm_name" {
  type    = string
  default = "ubuntu-base"
}

variable "output_directory" {
```

```

    type      = string
    default = "/var/lib/libvirt/images"
}

source "qemu" "ubuntu" {
    # Source ISO
    iso_url      = "https://cloud-images.ubuntu.com/releases/22.04/release/ubuntu-22.04-s
    iso_checksum = "file:https://cloud-images.ubuntu.com/releases/22.04/release/SHA256SUMS

    # Output configuration
    output_directory = var.output_directory
    vm_name          = "${var.vm_name}.qcow2"
    format           = "qcow2"

    # VM specifications
    memory          = 2048
    cpus            = 2
    disk_size       = "20G"
    disk_interface  = "virtio"
    net_device      = "virtio-net"

    # Cloud-init configuration
    cd_files = [
        "cloud-init/user-data",
        "cloud-init/meta-data"
    ]
    cd_label = "cidata"

    # SSH configuration
    ssh_username = "ubuntu"
    ssh_password = "ubuntu"
    ssh_timeout  = "20m"

    # Boot command (minimal for cloud images)
    boot_wait = "10s"

    # Don't start UI
    headless = true

    # Acceleration
    accelerator = "kvm"
}

build {
    sources = ["source.qemu.ubuntu"]
}

```

```

# Update system
provisioner "shell" {
  inline = [
    "sudo apt-get update",
    "sudo apt-get upgrade -y",
    "sudo apt-get install -y qemu-guest-agent cloud-init",
    "sudo systemctl enable qemu-guest-agent",
    "sudo apt-get clean"
  ]
}

# Install common tools
provisioner "shell" {
  inline = [
    "sudo apt-get install -y vim curl wget git htop net-tools",
    "sudo apt-get install -y python3-pip python3-venv",
    "sudo apt-get clean"
  ]
}

# Clean up for template use
provisioner "shell" {
  inline = [
    "sudo cloud-init clean",
    "sudo rm -rf /var/lib/cloud/instances/*",
    "sudo rm -f /etc/machine-id",
    "sudo touch /etc/machine-id",
    "sudo sync"
  ]
}
}

```

Cloud-Init Configuration

Create cloud-init files:

`~/labs/packer/cloud-init/user-data :`

```

#cloud-config
users:
  - name: ubuntu
    passwd: $6$rounds=4096$salt$hashedpassword # mkpasswd --method=sha-512
    lock_passwd: false
    sudo: ALL=(ALL) NOPASSWD:ALL

```

```
shell: /bin/bash
ssh_authorized_keys:
  - ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQBAQC... your-public-key

package_update: true
package_upgrade: true

packages:
  - qemu-guest-agent
  - cloud-init

runcmd:
  - systemctl enable qemu-guest-agent
  - systemctl start qemu-guest-agent
```

~/labs/packer/cloud-init/meta-data :

```
instance-id: ubuntu-base-001
local-hostname: ubuntu-base
```

Building Images

```
cd ~/labs/packer

# Initialize Packer (downloads plugins)
packer init ubuntu-base.pkr.hcl

# Validate configuration
packer validate ubuntu-base.pkr.hcl

# Build the image
packer build ubuntu-base.pkr.hcl

# Build with variables
packer build -var 'vm_name=ubuntu-22.04-base' ubuntu-base.pkr.hcl

# Build with debug output
packer build -debug ubuntu-base.pkr.hcl

# Verify the image
qemu-img info /var/lib/libvirt/images/ubuntu-base.qcow2
```

Specialized Base Images

Kerberos Base Image:

`~/labs/packer/kerberos-base.pkr.hcl` :

```
# Same source as ubuntu-base, but add:

build {
  sources = ["source.qemu.ubuntu"]

  provisioner "shell" {
    inline = [
      "sudo apt-get update",
      "sudo apt-get install -y krb5-kdc krb5-admin-server krb5-user",
      "sudo apt-get install -y libpam-krb5 libpam-ccreds",
      "sudo apt-get clean"
    ]
  }
}
```

Security Tools Base Image:

```
build {
  sources = ["source.qemu.ubuntu"]

  provisioner "shell" {
    inline = [
      "sudo apt-get update",
      "sudo apt-get install -y nmap wireshark-common tcpdump",
      "sudo apt-get install -y metasploit-framework",
      "sudo apt-get install -y john hashcat",
      "sudo apt-get clean"
    ]
  }
}
```

Packer Best Practices

1. Name your images clearly:

- `ubuntu-22.04-base-2025-02-10.qcow2`
- `kerberos-lab-base-v1.qcow2`

2. Version your images:

```
```hcl
variable "version" {
 default = "1.0.0"
}
```

```
vm_name = "${var.vm_name}-${var.version}.qcow2"
...`
```

### 1. Keep base images minimal:

- Only install software ALL VMs need
- Lab-specific software goes in Ansible

### 2. Compress images:

```
bash qemu-img convert -c -O qcow2 original.qcow2 compressed.qcow2
```

### 3. Test images before using in Terraform:

```
bash virt-install \ --name test-vm \ --ram 2048 \ --disk path=/var/lib/
libvirt/images/ubuntu-base.qcow2 \ --import \ --os-variant ubuntu22.04
```

---

# Ansible Integration

---

## Connecting Terraform to Ansible

### Method 1: Terraform Output → Ansible Inventory

kerberos-lab/outputs.tf :

```
output "vm_ips" {
 value = {
 for vm_name, vm in libvirt_domain.lab_vms :
 vm_name => vm.network_interface[0].addresses[0]
 }
}

output "ansible_inventory" {
 value = templatefile("${path.module}/templates/inventory.tpl", {
 vms = libvirt_domain.lab_vms
 })
}
```

kerberos-lab/templates/inventory.tpl :

```
[kdc]
${vms["kdc"].network_interface[0].addresses[0]} ansible_user=ubuntu

[clients]
${vms["client1"].network_interface[0].addresses[0]} ansible_user=ubuntu
${vms["client2"].network_interface[0].addresses[0]} ansible_user=ubuntu

[kerberos:children]
kdc
clients
```

### Generate inventory:

```
cd ~/labs/terraform/kerberos-lab
terraform output -raw ansible_inventory > ../../ansible/inventory/kerberos
```

### Method 2: Terraform Local Exec (Automatic)

Add to `main.tf` :

```
resource "null_resource" "ansible_provisioning" {
 depends_on = [libvirt_domain.lab_vms]

 provisioner "local-exec" {
 command = <<-EOT
 cd ../../ansible
 ansible-playbook -i inventory/kerberos playbooks/kerberos/site.yml
 EOT
 }

 # Trigger re-provisioning on IP changes
 triggers = {
 vm_ips = jsonencode([
 for vm in libvirt_domain.lab_vms :
 vm.network_interface[0].addresses[0]
])
 }
}
```

### Method 3: External Inventory Script (Dynamic)

~/labs/ansible/inventory/terraform.py :

```
#!/usr/bin/env python3
import json
import subprocess

def get_terraform_output(lab_name):
 cmd = f"terraform output -json -state=../../terraform/{lab_name}/terraform.tfstate"
 result = subprocess.run(cmd.split(), capture_output=True, text=True)
 return json.loads(result.stdout)

Parse Terraform state and generate Ansible inventory
Implementation details...
```

## Ansible Playbook Structure

```
~/labs/ansible/
├─ ansible.cfg
├─ inventory/
│ ├── kerberos
│ ├── ai-lab
│ └── cybersec
├─ playbooks/
│ ├── kerberos/
│ │ ├── site.yml
│ │ ├── kdc.yml
│ │ └── clients.yml
│ └── cybersec/
│ ├── site.yml
│ └── setup-tools.yml
├─ roles/
│ ├── kerberos-kdc/
│ ├── kerberos-client/
│ └── security-tools/
└─ group_vars/
 └─ all.yml
```

## Sample Ansible Playbook

~/labs/ansible/playbooks/kerberos/site.yml :

```

- name: Configure Kerberos Lab
 hosts: all
 become: yes

 tasks:
 - name: Wait for system to be ready
 wait_for_connection:
 timeout: 300

 - name: Update apt cache
 apt:
 update_cache: yes
 cache_valid_time: 3600

- name: Configure KDC
 hosts: kdc
 become: yes
 roles:
 - kerberos-kdc

- name: Configure Clients
 hosts: clients
 become: yes
 roles:
 - kerberos-client
```

---

## Lab Management Workflows

---

### Complete Lab Lifecycle

#### 1. Initial Build:

```
Build base image (once)
cd ~/labs/packer
packer build ubuntu-base.pkr.hcl

Deploy infrastructure
cd ~/labs/terraform/kerberos-lab
terraform init
```

```
terraform apply

Configure systems
cd ~/labs/ansible
ansible-playbook -i inventory/kerberos playbooks/kerberos/site.yml

Test
ansible -i inventory/kerberos all -m ping
```

## 2. Daily Work:

```
Start lab
cd ~/labs/terraform/kerberos-lab
for vm in $(virsh list --all --name | grep kerberos); do
 virsh start $vm
done

Do your testing...

Stop lab (save resources)
for vm in $(virsh list --name | grep kerberos); do
 virsh shutdown $vm
done
```

## 3. Rebuild After Failed Test:

```
cd ~/labs/terraform/kerberos-lab
terraform destroy -auto-approve && terraform apply -auto-approve
cd ~/labs/ansible
ansible-playbook -i inventory/kerberos playbooks/kerberos/site.yml
```

## 4. Rebuild After Successful Test (New Configuration):

```
Update Terraform config for new test
vim ~/labs/terraform/kerberos-lab/main.tf

Apply changes
terraform apply

Update Ansible if needed
vim ~/labs/ansible/playbooks/kerberos/site.yml
ansible-playbook -i inventory/kerberos playbooks/kerberos/site.yml
```

# Automation Scripts

## Master Rebuild Script:

`~/labs/scripts/rebuild-lab.sh :`

```
#!/bin/bash
set -e

LAB_NAME=$1

if [-z "$LAB_NAME"]; then
 echo "Usage: $0 <lab-name>"
 echo "Available labs: kerberos-lab, ai-lab, cybersecurity-lab, secdevops-lab"
 exit 1
fi

echo "=== Rebuilding $LAB_NAME ==="

Destroy infrastructure
cd ~/labs/terraform/$LAB_NAME
terraform destroy -auto-approve

Rebuild infrastructure
terraform apply -auto-approve

Wait for VMs to be ready
sleep 30

Configure with Ansible
cd ~/labs/ansible
ansible-playbook -i inventory/$LAB_NAME playbooks/${LAB_NAME}/site.yml

echo "=== $LAB_NAME rebuild complete ==="
```

## Quick Start Script:

`~/labs/scripts/start-lab.sh :`

```
#!/bin/bash
LAB_PREFIX=$1

if [-z "$LAB_PREFIX"]; then
 echo "Usage: $0 <lab-prefix>"
 echo "Example: $0 kerberos"
```

```
 exit 1
fi

echo "Starting all VMs with prefix: $LAB_PREFIX"
for vm in $(virsh list --all --name | grep "^${LAB_PREFIX}"); do
 echo "Starting $vm..."
 virsh start $vm
done

echo "Waiting for VMs to be ready..."
sleep 20

echo "VMs started:"
virsh list | grep "$LAB_PREFIX"
```

### Quick Stop Script:

`~/labs/scripts/stop-lab.sh :`

```
#!/bin/bash
LAB_PREFIX=$1

echo "Stopping all VMs with prefix: $LAB_PREFIX"
for vm in $(virsh list --name | grep "^${LAB_PREFIX}"); do
 echo "Shutting down $vm..."
 virsh shutdown $vm
done
```

Make executable:

```
chmod +x ~/labs/scripts/*.sh
```

Usage:

```
Start a lab
~/labs/scripts/start-lab.sh kerberos

Stop a lab
~/labs/scripts/stop-lab.sh kerberos

Rebuild a lab
~/labs/scripts/rebuild-lab.sh kerberos-lab
```

---

## Real-World Lab Examples

---

### Example 1: Kerberos Lab (4 VMs)

#### Architecture:

- 1 KDC (Kerberos Domain Controller)
- 1 Application Server
- 2 Clients

#### Directory Structure:

```
~/labs/terraform/kerberos-lab/
├─ main.tf
├─ variables.tf
├─ outputs.tf
└─ terraform.tfvars
```

#### main.tf :

```
terraform {
 required_providers {
 libvirt = {
 source = "dmacvicar/libvirt"
 version = "~> 0.7"
 }
 }
}

provider "libvirt" {
 uri = "qemu:///system"
}

Isolated network for Kerberos lab
resource "libvirt_network" "kerberos_net" {
 name = "kerberos_network"
 mode = "nat"
 domain = "kerberos.lab"
 addresses = ["192.168.100.0/24"]
 autostart = true

 dns {
```

```

 enabled = true
 local_only = true
 }
}

Define VMs
locals {
 vms = {
 kdc = {
 memory = 4096
 vcpu = 2
 ip = "192.168.100.10"
 }
 appserver = {
 memory = 2048
 vcpu = 2
 ip = "192.168.100.20"
 }
 client1 = {
 memory = 2048
 vcpu = 2
 ip = "192.168.100.30"
 }
 client2 = {
 memory = 2048
 vcpu = 2
 ip = "192.168.100.31"
 }
 }
}

Create volumes
resource "libvirt_volume" "kerberos_disks" {
 for_each = local.vms

 name = "kerberos-${each.key}.qcow2"
 pool = "default"
 source = "/var/lib/libvirt/images/ubuntu-base.qcow2"
 format = "qcow2"
}

Cloud-init for each VM
resource "libvirt_cloudinit_disk" "kerberos_init" {
 for_each = local.vms

 name = "kerberos-${each.key}-init.iso"

```

```

pool = "default"

user_data = templatefile("${path.module}/cloud-init-user-data.tpl", {
 hostname = "kerberos-${each.key}"
 fqdn = "kerberos-${each.key}.kerberos.lab"
})

network_config = templatefile("${path.module}/cloud-init-network-config.tpl", {
 ip_address = each.value.ip
})
}

Create VMs
resource "libvirt_domain" "kerberos_vms" {
 for_each = local.vms

 name = "kerberos-${each.key}"
 memory = each.value.memory
 vcpu = each.value.vcpu

 cloudinit = libvirt_cloudinit_disk.kerberos_init[each.key].id

 network_interface {
 network_id = libvirt_network.kerberos_net.id
 addresses = [each.value.ip]
 hostname = "kerberos-${each.key}"
 wait_for_lease = true
 }

 disk {
 volume_id = libvirt_volume.kerberos_disks[each.key].id
 }

 console {
 type = "pty"
 target_type = "serial"
 target_port = "0"
 }

 graphics {
 type = "spice"
 listen_type = "address"
 autoport = true
 }
}

```

**cloud-init-user-data.tpl :**

```
#cloud-config
hostname: ${hostname}
fqdn: ${fqdn}
manage_etc_hosts: true

users:
 - name: ubuntu
 sudo: ALL=(ALL) NOPASSWD:ALL
 groups: users, admin
 shell: /bin/bash
 ssh_authorized_keys:
 - ssh-rsa AAAAB3NzaC1yc2EAAA... # Your SSH key

package_update: true

packages:
 - qemu-guest-agent
 - vim
 - curl
 - net-tools

runcmd:
 - systemctl enable qemu-guest-agent
 - systemctl start qemu-guest-agent
```

**cloud-init-network-config.tpl :**

```
version: 2
ethernets:
 ens3:
 dhcp4: false
 addresses:
 - ${ip_address}/24
 gateway4: 192.168.100.1
 nameservers:
 addresses:
 - 192.168.100.1
 - 8.8.8.8
```

**outputs.tf :**

```

output "vm_ips" {
 value = {
 for vm_name, vm in libvirt_domain.kerberos_vms :
 vm_name => vm.network_interface[0].addresses[0]
 }
 description = "IP addresses of all Kerberos lab VMs"
}

output "kdc_ip" {
 value = libvirt_domain.kerberos_vms["kdc"].network_interface[0].addresses[0]
 description = "KDC IP address"
}

```

### Deploy:

```

cd ~/labs/terraform/kerberos-lab
terraform init
terraform apply

Get IPs
terraform output vm_ips

```

## Example 2: AI Lab (1-2 VMs with GPU)

**ai-lab/main.tf :**

```

terraform {
 required_providers {
 libvirt = {
 source = "dmacvicar/libvirt"
 version = "~> 0.7"
 }
 }
}

provider "libvirt" {
 uri = "qemu:///system"
}

AI Lab network
resource "libvirt_network" "ai_net" {
 name = "ai_network"
}

```

```

mode = "nat"
domain = "ai.lab"
addresses = ["192.168.102.0/24"]
autostart = true
}

Large disk for datasets
resource "libvirt_volume" "ai_disk" {
 name = "ai-workstation.qcow2"
 pool = "default"
 source = "/var/lib/libvirt/images/ubuntu-base.qcow2"
 format = "qcow2"
 size = 107374182400 # 100GB
}

High-memory VM for AI workloads
resource "libvirt_domain" "ai_workstation" {
 name = "ai-workstation"
 memory = 32768 # 32GB
 vcpu = 16 # 16 cores

 network_interface {
 network_id = libvirt_network.ai_net.id
 addresses = ["192.168.102.10"]
 wait_for_lease = true
 }

 disk {
 volume_id = libvirt_volume.ai_disk.id
 }

 # GPU passthrough (if available)
 # xml {
 # xslt = file("${path.module}/gpu-passthrough.xsl")
 # }
}

output "ai_workstation_ip" {
 value = libvirt_domain.ai_workstation.network_interface[0].addresses[0]
}

```

### Example 3: Cybersecurity Lab (4+ VMs)

**cybersecurity-lab/main.tf :**

```
terraform {
 required_providers {
 libvirt = {
 source = "dmacvicar/libvirt"
 version = "~> 0.7"
 }
 }
}

provider "libvirt" {
 uri = "qemu:///system"
}

Isolated network (no internet for vulnerable systems)
resource "libvirt_network" "cybersec_isolated" {
 name = "cybersec_isolated"
 mode = "none" # Completely isolated
 domain = "cybersec.lab"
 addresses = ["10.0.100.0/24"]
 autostart = true
}

Management network (with internet)
resource "libvirt_network" "cybersec_mgmt" {
 name = "cybersec_mgmt"
 mode = "nat"
 domain = "mgmt.cybersec.lab"
 addresses = ["192.168.103.0/24"]
 autostart = true
}

locals {
 vms = {
 attacker = {
 memory = 4096
 vcpu = 2
 ip_isolated = "10.0.100.10"
 ip_mgmt = "192.168.103.10"
 dual_nic = true
 }
 victim_web = {
 memory = 2048
 vcpu = 2
 ip_isolated = "10.0.100.20"
 ip_mgmt = null
 }
 }
}
```

```

 dual_nic = false
 }
 victim_db = {
 memory = 2048
 vcpu = 2
 ip_isolated = "10.0.100.21"
 ip_mgmt = null
 dual_nic = false
 }
 monitoring = {
 memory = 4096
 vcpu = 2
 ip_isolated = "10.0.100.30"
 ip_mgmt = "192.168.103.30"
 dual_nic = true
 }
}
}

resource "libvirt_volume" "cybersec_disks" {
 for_each = local.vms

 name = "cybersec-${each.key}.qcow2"
 pool = "default"
 source = "/var/lib/libvirt/images/ubuntu-base.qcow2"
 format = "qcow2"
}

resource "libvirt_domain" "cybersec_vms" {
 for_each = local.vms

 name = "cybersec-${each.key}"
 memory = each.value.memory
 vcpu = each.value.vcpu

 # Isolated network interface (always present)
 network_interface {
 network_id = libvirt_network.cybersec_isolated.id
 addresses = [each.value.ip_isolated]
 wait_for_lease = true
 }

 # Management network interface (if dual_nic)
 dynamic "network_interface" {
 for_each = each.value.dual_nic ? [1] : []
 content {

```

```

 network_id = libvirt_network.cybersec_mgmt.id
 addresses = [each.value.ip_mgmt]
 wait_for_lease = true
 }
}

disk {
 volume_id = libvirt_volume.cybersec_disks[each.key].id
}

output "attacker_mgmt_ip" {
 value = libvirt_domain.cybersec_vms["attacker"].network_interface[1].addresses[0]
 description = "Attacker management IP (for SSH access)"
}

```

## Example 4: SecDevOps Lab (2 VMs)

**secdevops-lab/main.tf :**

```

terraform {
 required_providers {
 libvirt = {
 source = "dmacvicar/libvirt"
 version = "~> 0.7"
 }
 }
}

provider "libvirt" {
 uri = "qemu:///system"
}

resource "libvirt_network" "secdevops_net" {
 name = "secdevops_network"
 mode = "nat"
 domain = "secdevops.lab"
 addresses = ["192.168.104.0/24"]
 autostart = true
}

locals {
 vms = {
 jenkins = {

```

```

 memory = 4096
 vcpu = 4
 ip = "192.168.104.10"
 disk_size = 53687091200 # 50GB
 }
 gitlab = {
 memory = 8192
 vcpu = 4
 ip = "192.168.104.20"
 disk_size = 107374182400 # 100GB
 }
}

resource "libvirt_volume" "secdevops_disks" {
 for_each = local.vms

 name = "secdevops-${each.key}.qcow2"
 pool = "default"
 source = "/var/lib/libvirt/images/ubuntu-base.qcow2"
 format = "qcow2"
 size = each.value.disk_size
}

resource "libvirt_domain" "secdevops_vms" {
 for_each = local.vms

 name = "secdevops-${each.key}"
 memory = each.value.memory
 vcpu = each.value.vcpu

 network_interface {
 network_id = libvirt_network.secdevops_net.id
 addresses = [each.value.ip]
 wait_for_lease = true
 }

 disk {
 volume_id = libvirt_volume.secdevops_disks[each.key].id
 }
}

output "jenkins_url" {
 value = "http://${libvirt_domain.secdevops_vms["jenkins"].network_interface[0].address}
}

```

```
output "gitlab_url" {
 value = "http://${libvirt_domain.secdevops_vms["gitlab"].network_interface[0].address}
}
```

---

## Troubleshooting

---

### Common Issues

#### 1. "Failed to connect to libvirt"

```
Check libvirtd service
sudo systemctl status libvirtd
sudo systemctl start libvirtd

Check permissions
groups $USER | grep libvirt
If not in group:
sudo usermod -aG libvirt $USER
newgrp libvirt
```

#### 2. "Network already exists"

```
List existing networks
virsh net-list --all

Destroy conflicting network
virsh net-destroy network_name
virsh net-undefine network_name

Re-run terraform
terraform apply
```

#### 3. "Volume already exists"

```
List volumes
virsh vol-list default

Delete conflicting volume
virsh vol-delete --pool default volume_name.qcow2
```

```
Or let Terraform import it
terraform import libvirt_volume.name default/volume_name.qcow2
```

#### 4. "Cloud-init not working"

```
Check if cloud-init ran
virsh console vm_name
Login and check:
cloud-init status

View cloud-init logs
sudo cat /var/log/cloud-init.log
sudo cat /var/log/cloud-init-output.log

Force re-run
sudo cloud-init clean --logs --reboot
```

#### 5. "VM won't get IP address"

```
Check DHCP leases
virsh net-dhcp-leases network_name

Check VM network interface
virsh domiflist vm_name

Restart networking in VM
virsh console vm_name
Login and:
sudo netplan apply
or
sudo systemctl restart systemd-networkd
```

#### 6. "Terraform state locked"

```
Find lock info
terraform force-unlock <lock-id>

Or remove .terraform.tfstate.lock.info manually (last resort)
rm .terraform.tfstate.lock.info
```

#### 7. "Packer build fails"

```
Enable debug mode
PACKER_LOG=1 packer build template.pkr.hcl

Check if QEMU is working
qemu-system-x86_64 --version

Check KVM acceleration
lsmmod | grep kvm
Should see: kvm_amd or kvm_intel
```

## 8. "Cannot allocate memory"

```
Check available memory
free -h

List all VMs and their memory
virsh list --all | tail -n +3 | awk '{print $2}' | while read vm; do
 echo -n "$vm: "
 virsh dominfo $vm | grep "Max memory"
done

Stop unused VMs
virsh shutdown unused_vm
```

## Validation Commands

Check everything is working:

```
Terraform
terraform version

Packer
packer version

Ansible
ansible --version

Libvirt
virsh version
virsh list --all

Networks
```

```
virsh net-list --all

Storage pools
virsh pool-list --all

Available images
virsh vol-list default
```

### Test connectivity:

```
From host to VM
ping -c 3 192.168.100.10

From VM to VM (via Ansible)
ansible -i inventory/kerberos all -m ping

SSH to VM
ssh ubuntu@192.168.100.10
```

---

## Cheat Sheet

---

### Terraform Quick Reference

```
Project lifecycle
terraform init # Initialize project
terraform validate # Validate syntax
terraform fmt # Format code
terraform plan # Preview changes
terraform apply # Apply changes
terraform destroy # Destroy everything

State management
terraform state list # List resources
terraform state show <resource> # Show resource details
terraform state rm <resource> # Remove from state
terraform import <resource> <id> # Import existing resource

Targeting
terraform apply -target=<resource> # Apply specific resource
terraform destroy -target=<resource> # Destroy specific resource
```

```
Variables
terraform apply -var="name=value" # Pass variable
terraform apply -var-file="prod.tfvars" # Use variable file

Output
terraform output # Show all outputs
terraform output <name> # Show specific output
terraform output -json # JSON format
```

## Packer Quick Reference

```
Build lifecycle
packer init template.pkr.hcl # Initialize/download plugins
packer validate template.pkr.hcl # Validate template
packer build template.pkr.hcl # Build image

Options
packer build -var 'name=value' # Pass variable
packer build -only=source.name # Build specific source
packer build -debug # Step through build
PACKER_LOG=1 packer build # Verbose logging

Inspection
packer inspect template.pkr.hcl # Show template details
```

## Ansible Quick Reference

```
Ad-hoc commands
ansible all -m ping # Test connectivity
ansible all -m shell -a "uptime" # Run command
ansible all -m apt -a "name=vim state=present" # Install package

Playbook execution
ansible-playbook site.yml # Run playbook
ansible-playbook site.yml --check # Dry run
ansible-playbook site.yml --diff # Show changes
ansible-playbook site.yml --tags "config" # Run specific tags
ansible-playbook site.yml --limit kdc # Limit to hosts

Inventory
```

```
ansible-inventory --list # Show inventory
ansible-inventory --graph # Show host groups
```

## Virsh Quick Reference

```
VM management
virsh list --all # List all VMs
virsh start <vm> # Start VM
virsh shutdown <vm> # Graceful shutdown
virsh destroy <vm> # Force power off
virsh undefine <vm> # Remove VM definition
virsh console <vm> # Access console
virsh dominfo <vm> # Show VM info

Network management
virsh net-list --all # List networks
virsh net-start <network> # Start network
virsh net-destroy <network> # Stop network
virsh net-undefine <network> # Remove network
virsh net-dhcp-leases <network> # Show DHCP leases

Storage management
virsh pool-list --all # List storage pools
virsh vol-list <pool> # List volumes in pool
virsh vol-delete <vol> --pool <p> # Delete volume
```

## Common Workflows

### Fresh lab deployment:

```
cd ~/labs/terraform/<lab-name>
terraform init && terraform apply -auto-approve
cd ~/labs/ansible
ansible-playbook -i inventory/<lab> playbooks/<lab>/site.yml
```

### Quick rebuild:

```
cd ~/labs/terraform/<lab-name>
terraform destroy -auto-approve && terraform apply -auto-approve && \
cd ~/labs/ansible && ansible-playbook -i inventory/<lab> playbooks/<lab>/site.yml
```

Add single VM:

```
Edit main.tf, add VM resource
terraform apply # Only new VM created
```

Remove single VM:

```
terraform destroy -target=libvirt_domain.<vm_name>
Or edit main.tf, remove resource, then:
terraform apply
```

# Resource Allocation Planning

## Your System: 32 Cores, 198GB RAM

Recommended allocation:

Lab	VMs	Cores/VM	RAM/VM	Total Cores	Total RAM
Kerberos	4	2	2GB	8	8GB
AI	1-2	8-16	16-32GB	16	32GB
Cybersecurity	4	2	2-4GB	8	12GB
SecDevOps	2	4	4-8GB	8	16GB
Total	11-12	-	-	40	68GB

Notes:

- You can run all labs simultaneously with these specs
- Leave 8 cores and 8GB for host system
- Total VM allocation: 40 cores (overcommit OK), 68GB RAM
- System overhead: Libvirt uses ~2GB additional RAM

Optimization strategies:

1. Stop unused labs to free resources

2. Use `count` or `for_each` to scale VMs down during development
  3. Reduce AI lab RAM if not actively training models
  4. Share Packer base images across labs (saves disk space)
- 

## Next Steps

---

1. **Start small:** Build Kerberos lab first
2. **Create base image:** Use Packer to create ubuntu-base.qcow2
3. **Deploy with Terraform:** Use examples above
4. **Configure with Ansible:** Create simple playbooks
5. **Document in Git:** Version control everything
6. **Iterate:** Add complexity as you gain confidence

## Portfolio Documentation

For each lab, document:

- Architecture diagram
- Terraform configuration
- Ansible playbooks
- Testing procedures
- Rebuild time (show efficiency)
- Lessons learned

This demonstrates:

- IaC expertise
  - Automation skills
  - Security knowledge
  - Operational excellence
- 

## Additional Resources

---

### Official Documentation:

- Terraform: <https://developer.hashicorp.com/terraform/docs>
- Packer: <https://developer.hashicorp.com/packer/docs>
- Ansible: <https://docs.ansible.com/>
- Libvirt: <https://libvirt.org/docs.html>

### **Terraform Libvirt Provider:**

- <https://github.com/dmacvicar/terraform-provider-libvirt>
- <https://registry.terraform.io/providers/dmacvicar/libvirt/latest/docs>

### **Community:**

- r/Terraform
  - r/ansible
  - r/homelab
- 

## **Conclusion**

---

This guide provides the **20% of knowledge that will handle 80% of your lab management needs**. Focus on:

1.  Packer for creating base images
2.  Terraform for infrastructure provisioning
3.  Ansible for configuration management
4.  Git for version control
5.  Scripts for automation

**Your advantage:** 20 years of Linux/virtualization experience means you'll master this stack faster than most. The concepts will be familiar, only the tools are new.

**Time investment:** 2-3 days to master basics, 1-2 weeks to become proficient.

**ROI:** Rebuild labs in minutes instead of hours. Impressive portfolio material. Highly marketable skills.

---

**Document Version:** 1.0

**Last Updated:** February 2026

**Author:** Infrastructure automation guide for experienced Linux/KVM administrators

**License:** Use freely for personal projects and portfolio development